Table des matières

Labo 03 OLTP — Écrire dans la base transactionnelle 1
Le terrain, en une minute 2
Partie 1 — Le bac à sable 2
Partie 2 — Créer une table et ses types 3
Partie 3 — Les contraintes 4
Partie 4 — INSERT 6
Partie 5 — UPDATE et DELETE 8
Partie 6 — MERGE et l'upsert 9
Partie 7 — Les vues 11
Partie 8 — Ce qui s'annule 12
Partie 9 — Ce que le contraste vous a montré 13
Nettoyage 14
Et ensuite ? 15

Labo 03 OLTP — Écrire dans la base transactionnelle

Laboratoire du Cours 03, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On refait les gestes du laboratoire OLAP sur l'autre terrain, la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Tout ce que le labo crée vit dans un schéma à vous, `labo03` ; les tables d'AdventureWorks ne servent qu'à être lues et copiées. Comptez une heure et demie.

Les deux voies — Ce labo est le jumeau de `Labo_03_OLAP_SQL.md`. Les gestes sont les mêmes ; les garanties du moteur, non. Plusieurs commandes qui passaient en silence sur l'entrepôt sont refusées ici, et l'inverse arrive aussi : notez chaque écart, la Partie 9 les rassemble.

Une commande à la fois — Plusieurs étapes contiennent des erreurs voulues. Lancez les commandes une par une plutôt que tout le bloc d'un coup. Quand une étape ouvre une transaction avec `BEGIN`, allez jusqu'au `ROLLBACK` indiqué avant de passer à la suite ; si vous vous perdez, un `ROLLBACK;` seul ne fait jamais de mal.

© 2026 iAlexMG. Tous droits réservés.

Le terrain, en une minute

Table	Ce qu'elle contient	Grain	Ce qui servira ici
`sales.specialoffer`	les promotions	une promotion, 16 lignes	la source qu'on copie
`sales.specialofferproduct`	les produits visés par chaque promotion	un couple promotion et produit, 538 lignes	la source d'une table liée par clé étrangère
`production.product`	le catalogue	un produit, 504 lignes	les casques, sous-catégorie 31

Rien dans ce labo n'écrit dans ces trois tables : on les lit, on les copie, et on travaille sur les copies.

Partie 1 — Le bac à sable

1.1 — Un schéma à soi

Théorie : `Theorie_03_SQL.md`, section 1.

Objectif : créer l'espace de travail du labo.

```sql
CREATE SCHEMA labo03;
```

Observez. PostgreSQL répond `CREATE SCHEMA`. Tout ce qui suit s'écrit `labo03.nom_de_table`, et la section Nettoyage supprimera le schéma avec tout son contenu, d'une seule commande.

1.2 — Copier une table, et voir ce que la copie a perdu

Théorie : `Theorie_03_SQL.md`, sections 1 et 2.

Objectif : fabriquer une copie de travail des promotions.

```sql
CREATE TABLE labo03.promotion_brute AS
SELECT * FROM sales.specialoffer;
```

Observez. `SELECT 16` : la copie existe, avec ses 16 promotions. Comparez maintenant les contraintes de l'original et de la copie, dans le catalogue `pg_constraint`, où PostgreSQL range les siennes :

```sql
SELECT conrelid::regclass AS la_table, conname, contype
FROM pg_constraint
WHERE conrelid IN ('sales.specialoffer'::regclass, 'labo03.promotion_brute'::regclass)
  AND contype IN ('p', 'c')
ORDER BY 1, 2;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Cinq lignes, toutes pour `sales.specialoffer` : la clé primaire (`p`) et quatre `CHECK` (`c`), dont `CK_SpecialOffer_EndDate`, qui interdit une fin avant le début. La copie n'en a aucune. Même constat pour les colonnes obligatoires et les valeurs par défaut :

```
SELECT table_schema, column_name, is_nullable, column_default
FROM information_schema.columns
WHERE (table_schema, table_name) IN (('sales', 'specialoffer'), ('labo03', 'promotion_brute'))
  AND column_name IN ('specialofferid', 'discountpct', 'minqty', 'modifieddate')
ORDER BY column_name, table_schema;
```

Observez. Côté `sales`, les quatre colonnes sont obligatoires (`NO`) et ont une valeur par défaut, dont le `nextval(…)` qui génère les numéros de promotion. Côté `labo03`, tout est facultatif (`YES`) et aucune valeur par défaut n'a survécu. La copie a gardé les données et les types, rien de ce qui les protégeait.

Partie 2 — Créer une table et ses types

2.1 — Ce que le type fait à la valeur

Théorie : `Theorie_03_SQL.md`, section 2.

Objectif : voir, valeur par valeur, ce qu'un type refuse et ce qu'il transforme.

```
CREATE TABLE labo03.essai_type (
    montant  NUMERIC(6,2),
    code     VARCHAR(3),
    quantite SMALLINT
);

INSERT INTO labo03.essai_type (montant)  VALUES (12.345);
INSERT INTO labo03.essai_type (montant)  VALUES (12345.6);
INSERT INTO labo03.essai_type (code)     VALUES ('ABCD');
INSERT INTO labo03.essai_type (quantite) VALUES (40000);
INSERT INTO labo03.essai_type (code)     VALUES ('AB   ');
INSERT INTO labo03.essai_type (quantite) VALUES (12.6);
```

Observez. Trois refus : « champ numérique en dehors des limites » pour 12 345,6, « valeur trop longue pour le type character varying(3) » pour `ABCD`, « smallint en dehors des limites » pour 40 000. Et trois acceptations, qu'il faut regarder de près :

```
SELECT montant, code, LENGTH(code) AS longueur, quantite
FROM labo03.essai_type;
```

Observez. 12,345 est devenu 12,35. `'AB   '`, cinq caractères, est entré dans un `VARCHAR(3)` : les espaces en trop ont été coupés sans erreur, parce que ce n'étaient que des espaces, et il reste `AB`, de longueur 3. 12,6 est devenu 13 dans la colonne entière. Aucune de ces trois transformations n'a laissé de trace à l'écran.

© 2026 iAlexMG. Tous droits réservés.

2.2 — Une table faite pour durer

Théorie : `Theorie_03_SQL.md`, sections 2 et 3.

Objectif : créer une vraie table de promotions, avec ses types et ses contraintes, puis la remplir.

```
CREATE TABLE labo03.promotion (
    specialofferid INTEGER      GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    description    VARCHAR(255) NOT NULL,
    discountpct    NUMERIC(4,2) NOT NULL DEFAULT 0 CHECK (discountpct BETWEEN 0 AND 1),
    type           VARCHAR(50)  NOT NULL,
    category       VARCHAR(50)  NOT NULL,
    startdate      DATE         NOT NULL,
    enddate        DATE         NOT NULL,
    minqty         INTEGER      NOT NULL DEFAULT 0 CHECK (minqty >= 0),
    maxqty         INTEGER      CHECK (maxqty >= minqty),
    CHECK (enddate >= startdate)
);

INSERT INTO labo03.promotion (specialofferid, description, discountpct, type, category,
                              startdate, enddate, minqty, maxqty)
SELECT specialofferid, description, discountpct, type, category,
       startdate, enddate, minqty, maxqty
FROM sales.specialoffer;
```

Observez. `INSERT 0 16`. Trois choix de type méritent un mot. Le rabais est une fraction, 0,15 pour 15 %, d'où `NUMERIC(4,2)` et un `CHECK` entre 0 et 1. Les dates de la source sont des horodatages toujours à minuit (vous l'avez constaté au Cours 02 sur les commandes), d'où `DATE`. Et les numéros existants sont chargés tels quels, ce que permet `GENERATED BY DEFAULT`. Vérifiez le résultat :

```
SELECT specialofferid, discountpct, startdate, maxqty
FROM labo03.promotion
ORDER BY specialofferid
LIMIT 3;
```

Observez. La promotion 1 a un rabais de 0,00 et pas de plafond ; les promotions 2 et 3, 0,02 et 0,05, avec des plafonds de 14 et 24 unités. Les horodatages sont devenus des dates à l'insertion, sans perte, et les rabais à quatre décimales de la source tiennent en deux.

Partie 3 — Les contraintes

3.1 — Violer chaque contrainte, une à la fois

Théorie : `Theorie_03_SQL.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

Objectif : lire le message de chaque contrainte, et constater que rien n'est écrit.

```sql
-- clé primaire : le numéro 1 existe déjà
INSERT INTO labo03.promotion (specialofferid, description, type, category, startdate, enddate)
VALUES (1, 'Doublon', 'Seasonal Discount', 'Customer', '2025-06-01', '2025-06-30');

-- NOT NULL : pas de catégorie
INSERT INTO labo03.promotion (specialofferid, description, type, category, startdate, enddate)
VALUES (17, 'Sans catégorie', 'Seasonal Discount', NULL, '2025-06-01', '2025-06-30');

-- CHECK : la fin avant le début
INSERT INTO labo03.promotion (specialofferid, description, type, category, startdate, enddate)
VALUES (17, 'Dates inversées', 'Seasonal Discount', 'Customer', '2025-06-30', '2025-06-01');

-- CHECK : 15 au lieu de 0,15
INSERT INTO labo03.promotion (specialofferid, description, discountpct, type, category, startdate, enddate)
VALUES (17, 'Quinze pour cent', 15, 'Seasonal Discount', 'Customer', '2025-06-01', '2025-06-30');

-- CHECK : un plafond sous le plancher
INSERT INTO labo03.promotion (specialofferid, description, type, category, startdate, enddate, minqty, maxqty)
VALUES (17, 'Plafond inférieur', 'Volume Discount', 'Reseller', '2025-06-01', '2025-06-30', 20, 10);
```

Observez. Cinq refus, cinq messages : « la valeur d'une clé dupliquée rompt la contrainte unique « promotion_pkey » », « une valeur NULL viole la contrainte NOT NULL de la colonne « category » », puis trois fois « la nouvelle ligne de la relation « promotion » viole la contrainte de vérification », suivie du nom de la contrainte : `promotion_check1`, `promotion_discountpct_check` et `promotion_check`. Le premier et le dernier ne disent rien de ce qu'ils vérifient : il faut relire la définition de la table pour apprendre que `promotion_check1` porte sur les dates et `promotion_check` sur le plafond, et c'est la meilleure raison de nommer ses contraintes soi-même. Un rabais de 150 aurait levé une autre erreur encore, avant même le `CHECK` : `NUMERIC(4,2)` ne va pas au-delà de 99,99.

```sql
SELECT COUNT(*) FROM labo03.promotion;
```

Observez. Toujours 16 : aucune des cinq commandes n'a écrit quoi que ce soit.

3.2 — `CHECK` et `NULL`

Théorie : `Theorie_03_SQL.md`, section 3.

Objectif : compter les promotions qui passent le `CHECK` du plafond sans avoir de plafond.

```sql
SELECT COUNT(*) AS promotions, COUNT(maxqty) AS avec_plafond
FROM labo03.promotion;
```

Observez. 16 promotions, 4 plafonds. Les 12 autres ont un `maxqty` à `NULL`, et la condition `maxqty >= minqty` n'est ni vraie ni fausse pour elles : le `CHECK` les a laissées passer. Ici, c'est voulu, une promotion sans plafond est légitime. Si le plafond devait être obligatoire, il faudrait un `NOT NULL`.

© 2026 iAlexMG. Tous droits réservés.

3.3 — La clé étrangère, dans les deux sens

Théorie : `Theorie_03_SQL.md`, section 3.

Objectif : lier les produits aux promotions, puis tenter de casser le lien des deux côtés.

```
CREATE TABLE labo03.promotion_produit (
    specialofferid INTEGER NOT NULL REFERENCES labo03.promotion (specialofferid),
    productid      INTEGER NOT NULL REFERENCES production.product (productid),
    PRIMARY KEY (specialofferid, productid)
);

INSERT INTO labo03.promotion_produit (specialofferid, productid)
SELECT specialofferid, productid
FROM sales.specialofferproduct;
```

Observez. `INSERT 0 538`. La table de liaison a deux clés étrangères, une vers vos promotions, une vers le vrai catalogue, et une clé primaire sur le couple. Tentez d'y inscrire une promotion qui n'existe pas, puis de supprimer une promotion qui a des produits :

```
INSERT INTO labo03.promotion_produit (specialofferid, productid) VALUES (99, 707);

DELETE FROM labo03.promotion WHERE specialofferid = 16;
```

Observez. Le premier : « … viole la contrainte de clé étrangère « promotion_produit_specialofferid_fkey » », et le détail précise que la clé 99 n'est pas présente dans la table `promotion`. Le second : la clé 16 « est toujours référencée à partir de la table « promotion_produit » ». La même contrainte protège les deux tables.

Partie 4 — `INSERT`

4.1 — La clé générée, et le compteur en retard

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : ajouter une promotion sans fournir son numéro, et récupérer celui que le moteur lui donne.

```
INSERT INTO labo03.promotion (description, type, category, startdate, enddate)
VALUES ('Soldes de la rentrée', 'Seasonal Discount', 'Customer', '2025-08-15', '2025-09-15')
RETURNING specialofferid;
```

Observez. Erreur : la clé 1 existe déjà. Les 16 promotions ont été chargées avec leurs numéros, et le compteur d'identité, qui ne les a pas vues passer, commence à 1. Relancez la même commande : erreur encore, avec la clé 2. La tentative ratée a consommé le numéro 1 pour de bon. Remettez le compteur au-delà du plus grand numéro, lu dans la table plutôt que recopié à la main, puis réessayez :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT setval(pg_get_serial_sequence('labo03.promotion', 'specialofferid'), MAX(specialofferid))
FROM labo03.promotion;

INSERT INTO labo03.promotion (description, type, category, startdate, enddate)
VALUES ('Soldes de la rentrée', 'Seasonal Discount', 'Customer', '2025-08-15', '2025-09-15')
RETURNING specialofferid, discountpct, minqty, maxqty;
```

Observez. Le `setval` rend 16, le plus grand numéro chargé. Puis une ligne revient : 17, 0,00, 0 et un plafond vide. `RETURNING` a rendu le numéro généré, et les valeurs par défaut de `discountpct` et `minqty` se sont appliquées aux colonnes omises. `ALTER COLUMN specialofferid RESTART WITH 17` aurait fait le même travail avec un nombre tapé à la main, juste aujourd'hui et faux au prochain chargement.

4.2 — L'`INSERT` sans liste de colonnes

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : voir une inversion de colonnes passer sans erreur.

```sql
INSERT INTO labo03.promotion
VALUES (18, 'Liquidation des casques', 0.25, 'Customer', 'Discontinued Product',
        '2025-09-01', '2025-09-30', 0, NULL);

SELECT specialofferid, type, category
FROM labo03.promotion
WHERE specialofferid = 18;
```

Observez. `INSERT 0 1`, puis un `type` qui vaut `Customer` et une `category` qui vaut `Discontinued Product`. L'auteur a écrit la catégorie avant le type ; les deux colonnes sont du texte, et le moteur a rangé les valeurs par position, sans rien vérifier d'autre. Avec une liste de colonnes, l'erreur aurait sauté aux yeux dès l'écriture.

4.3 — `INSERT … SELECT`

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : rattacher tous les casques à la promotion de la rentrée, en une commande.

```sql
INSERT INTO labo03.promotion_produit (specialofferid, productid)
SELECT 17, p.productid
FROM production.product AS p
WHERE p.productsubcategoryid = 31
RETURNING specialofferid, productid;
```

Observez. Trois lignes, les produits 707, 708 et 711, les trois casques Sport-100. La constante 17 est répétée sur chaque ligne produite par le `SELECT`.

© 2026 iAlexMG. Tous droits réservés.

Partie 5 — UPDATE et DELETE

5.1 — Compter, écrire, vérifier

Théorie : `Theorie_03_SQL.md`, sections 5 et 8.

Objectif : porter le rabais de la promotion 2 à 3 %, et rattraper l'oubli du `WHERE`.

```
SELECT COUNT(*) FROM labo03.promotion WHERE specialofferid = 2;

BEGIN;
UPDATE labo03.promotion SET discountpct = 0.03;
```

Observez. Le compte préalable annonce 1 ligne, le compte rendu en déclare 18 : le `WHERE` est resté dans le `SELECT`. Toutes les promotions viennent de passer à 3 %, mais la transaction est encore ouverte. Annulez, puis refaites la commande correctement :

```
ROLLBACK;

UPDATE labo03.promotion SET discountpct = 0.03 WHERE specialofferid = 2;
```

Observez. `UPDATE 1`, le compte attendu. Sans le `BEGIN`, la première commande aurait été validée à la seconde où elle a réussi.

5.2 — UPDATE … FROM et la source en double

Théorie : `Theorie_03_SQL.md`, section 5.

Objectif : appliquer une liste de rabais qui contient, par erreur, deux valeurs pour la promotion 3.

```
UPDATE labo03.promotion AS p
SET discountpct = s.rabais
FROM (VALUES (3, 0.06), (3, 0.07)) AS s (id, rabais)
WHERE p.specialofferid = s.id;

SELECT specialofferid, discountpct
FROM labo03.promotion
WHERE specialofferid IN (2, 3)
ORDER BY specialofferid;
```

Observez. `UPDATE 1`, aucune erreur, et un rabais de 0,07 pour la promotion 3 lors de notre essai. La théorie montrait la première valeur l'emporter, ici c'est la seconde : le moteur prend celle qu'il rencontre, et rien ne dit laquelle ce sera. `(VALUES …) AS s (id, rabais)` fabrique une petite table à la volée, pratique pour un essai.

5.3 — DELETE et TRUNCATE

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_03_SQL.md`, section 5.

Objectif : retirer deux promotions, puis tenter de vider des tables entières.

```
DELETE FROM labo03.promotion WHERE specialofferid = 18;
DELETE FROM labo03.promotion WHERE specialofferid = 17;
TRUNCATE labo03.promotion;
```

Observez. La promotion 18, mal saisie en partie 4, part : `DELETE 1`. La 17 résiste, parce que les trois casques y sont rattachés. Et `TRUNCATE` refuse de vider une table référencée par une clé étrangère : « ne peut pas tronquer une table référencée dans une contrainte de clé étrangère ». Sur la copie brute, qui n'a aucune contrainte, rien ne l'arrête :

```
BEGIN;
TRUNCATE labo03.promotion_brute;
SELECT COUNT(*) FROM labo03.promotion_brute;
ROLLBACK;
SELECT COUNT(*) FROM labo03.promotion_brute;
```

Observez. 0 ligne, puis 16 après le `ROLLBACK` : sur PostgreSQL, même `TRUNCATE` s'annule. Retirez enfin la promotion 17 dans le bon ordre, les enfants avant le parent :

```
DELETE FROM labo03.promotion_produit WHERE specialofferid = 17;
DELETE FROM labo03.promotion WHERE specialofferid = 17;
SELECT COUNT(*) FROM labo03.promotion;
```

Observez. `DELETE 3`, `DELETE 1`, et 16 promotions : le compte de départ.

Partie 6 — `MERGE` et l'upsert

6.1 — Fusionner une mise à jour

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : prolonger une liquidation et en créer une nouvelle, d'une seule commande.

```
CREATE TABLE labo03.promotion_maj (
    specialofferid INTEGER,
    description    VARCHAR(255),
    discountpct    NUMERIC(4,2),
    type           VARCHAR(50),
    category       VARCHAR(50),
    startdate      DATE,
    enddate        DATE
```


© 2026 iAlexMG. Tous droits réservés.

```
);

INSERT INTO labo03.promotion_maj (specialofferid, description, discountpct, type, category, startdate, enddate)
VALUES (16, 'Mountain-500 Silver Clearance Sale', 0.45, 'Discontinued Product', 'Reseller', '2025-03-30', '2025-06-29'),
       (20, 'Road-250 Clearance Sale',            0.30, 'Discontinued Product', 'Reseller', '2025-06-01', '2025-07-31');

MERGE INTO labo03.promotion AS p
USING labo03.promotion_maj AS m
   ON m.specialofferid = p.specialofferid
WHEN MATCHED THEN
    UPDATE SET discountpct = m.discountpct, enddate = m.enddate
WHEN NOT MATCHED THEN
    INSERT (specialofferid, description, discountpct, type, category, startdate, enddate)
    VALUES (m.specialofferid, m.description, m.discountpct, m.type, m.category, m.startdate, m.enddate)
RETURNING merge_action(), p.specialofferid, p.discountpct, p.enddate;
```

Observez. Deux lignes : `UPDATE` pour la promotion 16, passée à 0,45 jusqu'au 29 juin 2025, `INSERT` pour la 20, à 0,30. Puis `MERGE 2`. Le `RETURNING` d'un `MERGE` et sa fonction `merge_action()` sont arrivés avec PostgreSQL 17, d'après ses notes de version.

6.2 — Rejouer

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : vérifier qu'un chargement relancé ne crée pas de doublon.

Relancez exactement le même `MERGE`, puis comptez :

```
SELECT COUNT(*) FROM labo03.promotion;
```

Observez. Le `RETURNING` affiche cette fois deux `UPDATE`, et la table compte 17 promotions, pas 18. La promotion 20 existe désormais, elle tombe dans `WHEN MATCHED`. Un chargement qui tourne deux fois par erreur laisse la table dans le même état qu'une seule fois.

6.3 — La source en double

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : voir `MERGE` refuser ce que l'`UPDATE … FROM` de la partie 5 acceptait.

```
INSERT INTO labo03.promotion_maj (specialofferid, description, discountpct, type, category, startdate, enddate)
VALUES (16, 'Mountain-500 Silver Clearance Sale', 0.50, 'Discontinued Product', 'Reseller', '2025-03-30', '2025-06-29');
```

Relancez le `MERGE` de la sous-partie 6.1.

© 2026 iAlexMG. Tous droits réservés.

Observez. « MERGE command cannot affect row a second time », et l'astuce en anglais aussi : assurez-vous qu'une seule ligne de la source corresponde à chaque ligne de la cible. La promotion 16 a maintenant deux lignes dans la source, à 0,45 et à 0,50, et le moteur refuse de choisir. La commande échoue en entier : même la mise à jour de la promotion 20, qui était sans reproche, n'a pas eu lieu.

6.4 — L'upsert de PostgreSQL

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : écrire le même besoin avec `ON CONFLICT`.

```
INSERT INTO labo03.promotion (specialofferid, description, discountpct, type, category, startdate, enddate)
VALUES (16, 'Mountain-500 Silver Clearance Sale', 0.40, 'Discontinued Product', 'Reseller', '2025-03-30', '2025-05-29')
ON CONFLICT (specialofferid) DO UPDATE
    SET discountpct = EXCLUDED.discountpct,
        enddate     = EXCLUDED.enddate
RETURNING specialofferid, discountpct, enddate;
```

Observez. La promotion 16 revient à 0,40 jusqu'au 29 mai 2025, ses valeurs d'origine. La ligne proposée est entrée en conflit avec la clé primaire, et la branche `DO UPDATE` a pris le relais. Le compte rendu dit pourtant `INSERT 0 1` : c'est la commande qui s'appelle `INSERT`, pas ce qu'elle a fait.

Partie 7 — Les vues

7.1 — Une vue suit les données

Théorie : `Theorie_03_SQL.md`, sections 7 et 8.

Objectif : publier la liste des promotions actives, et la voir se vider sous l'effet d'une erreur.

```
CREATE VIEW labo03.promotion_active AS
SELECT specialofferid, description, discountpct
FROM labo03.promotion
WHERE discountpct > 0;

SELECT COUNT(*) FROM labo03.promotion_active;
```

Observez. 16 promotions actives sur 17 : seule la promotion 1, `No Discount`, n'a pas de rabais. Simulez maintenant une erreur de saisie, dans une transaction :

```
BEGIN;
UPDATE labo03.promotion SET discountpct = 0;
SELECT COUNT(*) FROM labo03.promotion_active;
ROLLBACK;
SELECT COUNT(*) FROM labo03.promotion_active;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. `UPDATE 17`, puis une vue vide, puis 16 à nouveau après le `ROLLBACK`. Personne n'a touché à la vue : elle a simplement relu la table. Un tableau de bord branché dessus aurait annoncé qu'aucune promotion n'était en cours, avec la même assurance que d'habitude.

7.2 — La vue `SELECT *` qui ne voit pas la nouvelle colonne

Théorie : `Theorie_03_SQL.md`, section 7.

Objectif : ajouter une colonne à la table et regarder ce que la vue en fait.

```
CREATE VIEW labo03.promotion_toute AS
SELECT * FROM labo03.promotion;

ALTER TABLE labo03.promotion ADD COLUMN canal VARCHAR(20) NOT NULL DEFAULT 'tous';

SELECT * FROM labo03.promotion_toute WHERE specialofferid = 1;
SELECT canal, COUNT(*) FROM labo03.promotion GROUP BY canal;
```

Observez. La vue affiche neuf colonnes, de `specialofferid` à `maxqty`, sans `canal`. La table, elle, a bien sa dixième colonne, remplie de « tous » sur les 17 lignes. PostgreSQL a remplacé l'étoile par la liste des colonnes au moment de créer la vue, et cette liste ne bouge plus.

7.3 — Supprimer une table dont d'autres objets dépendent

Théorie : `Theorie_03_SQL.md`, section 7.

Objectif : lire ce qu'un `DROP` emporterait, sans rien emporter.

```
DROP TABLE labo03.promotion;
```

Observez. Refus, et un détail qui énumère trois dépendants : la contrainte de clé étrangère de `promotion_produit`, et les deux vues. C'est exactement la liste de ce qu'un `DROP TABLE … CASCADE` supprimerait. Ne le lancez pas : la table sert encore.

Partie 8 — Ce qui s'annule

8.1 — Un DDL au milieu d'une transaction

Théorie : `Theorie_03_SQL.md`, section 8.

Objectif : vérifier que PostgreSQL annule une création de table avec le reste.

```
BEGIN;
UPDATE labo03.promotion SET discountpct = 0.45 WHERE specialofferid = 16;
CREATE TABLE labo03.essai_ddl (x INTEGER);
ROLLBACK;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT discountpct FROM labo03.promotion WHERE specialofferid = 16;
SELECT to_regclass('labo03.essai_ddl') AS essai_ddl;
```

Observez. Le rabais est resté à 0,40, et `to_regclass`, qui rend le nom d'une table si elle existe, rend une valeur vide : la table n'a jamais existé. DuckDB a fait de même dans le labo jumeau : sur les deux terrains de la formation, le DDL s'annule avec le reste.

8.2 — Même un `DROP`

Théorie : `Theorie_03_SQL.md`, section 8.

Objectif : supprimer une table, puis la récupérer.

```sql
BEGIN;
DROP TABLE labo03.promotion_brute;
SELECT to_regclass('labo03.promotion_brute') AS promotion_brute;
ROLLBACK;
SELECT COUNT(*) FROM labo03.promotion_brute;
```

Observez. La table disparaît, puis revient avec ses 16 lignes. Tant que la transaction est ouverte, PostgreSQL garde de quoi tout défaire. Ce filet ne vaut que si vous avez tapé `BEGIN` avant : hors transaction, le `DROP` est validé à la seconde où il réussit.

Partie 9 — Ce que le contraste vous a montré

Théorie : `Theorie_03_SQL.md`, section 9.

Ce qui a changé	Voie OLAP (DuckDB)	Voie OLTP (PostgreSQL)
Le bac à sable	le préfixe `LABO03_` dans `dbo`	le schéma `labo03`
Recréer une table	`CREATE OR REPLACE TABLE`, sans prévenir	`DROP TABLE` puis `CREATE TABLE`
`VARCHAR(3)` et `ABCD`	accepté en entier, la longueur n'est pas vérifiée	refusé
`SMALLINT` et 40 000	refusé	refusé
Un doublon de clé primaire	refusé	refusé
Une clé étrangère vers une ligne absente	refusée	refusée
Supprimer une ligne encore référencée	refusé	refusé
`CHECK`	appliqué à chaque ligne	appliqué à chaque ligne

© 2026 iAlexMG. Tous droits réservés.

Ce qui a changé	Voie OLAP (DuckDB)	Voie OLTP (PostgreSQL)
Le compteur de la clé générée	une séquence à part, appelée par la valeur par défaut	une colonne d'identité
Le compteur en retard	une erreur de clé dupliquée, et un numéro perdu par ligne visée	une erreur de clé dupliquée, et le numéro tenté perdu
Recaler le compteur	détacher la séquence, la recréer, la rattacher, avec un nombre écrit en toutes lettres	`setval`, calculé sur le plus grand numéro de la table
La clé générée	rendue par `RETURNING`	rendue par `RETURNING`
`INSERT` sans liste de colonnes	refusé quand les valeurs ne couvrent pas la table	accepté, par position
`UPDATE … FROM` sur une source en double	accepté, une valeur au hasard	accepté, une valeur au hasard
`MERGE` sur une source en double	accepté, une valeur au hasard	refusé
L'upsert	`MERGE` ou `ON CONFLICT`	`MERGE` ou `ON CONFLICT`
Une vue `SELECT *` après un ajout de colonne	elle suit la table et montre la colonne	figée sur ses colonnes d'origine
`DROP TABLE` sous une vue	accepté, la vue casse	refusé, avec la liste des dépendants
Un `CREATE TABLE` après un `UPDATE`, puis `ROLLBACK`	tout est annulé	tout est annulé

Trois leçons à emporter. La première : les deux terrains appliquent les mêmes contraintes, et ce n'est pas là que les garanties divergent. Elles divergent sur les types, sur la fusion et sur les vues : un `VARCHAR(3)` qui garde quatre caractères, un `MERGE` qui accepte une source en double, une vue qui change de forme avec sa table. Un script qui passe sur l'un peut échouer sur l'autre, dans les deux sens. La deuxième : les erreurs d'écriture les plus coûteuses ne lèvent rien. Le `WHERE` oublié, l'`INSERT` par position, la source en double d'un `UPDATE … FROM` passent sur les deux moteurs, et celle d'un `MERGE` passe sur DuckDB ; seul le compte rendu les trahit, à condition de le lire. La troisième : sur les deux terrains, une transaction ouverte à temps annule tout, DDL compris. Ce n'est pas vrai de tous les entrepôts : d'après sa documentation, Snowflake valide d'office la transaction en cours avant un DDL, et c'est son voyage dans le temps, au Cours 11, qui tient le rôle du filet, après coup.

Nettoyage

```sql
DROP SCHEMA labo03 CASCADE;
```

© 2026 iAlexMG. Tous droits réservés.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_03_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vous savez maintenant créer, remplir, corriger et fusionner, et vous savez ce que chaque moteur promet quand il accepte une commande. Le Cours 04 revient à la lecture, avec l'outil le plus demandé en entretien d'analytique : les fonctions de fenêtrage, qui calculent sur un groupe de lignes sans les écraser en une seule.

© 2026 iAlexMG. Tous droits réservés.